PRACTICAL REPORT 14

Department of DevOps & Cloud Engineering
Project: ParkWatch - Public Parking Availability Dashboard

Student: Sajiya Shaikh | **Roll No:** 24102A2005
Academic Year: 2026 | **Milestone:** Task 14

Task 14 Report: Automated Disaster Recovery & Rollback Scripts

Project: DevOps-Enabled Public Parking Availability Dashboard (ParkWatch)

Student: Sajiya Shaikh

Roll Number: 24102A2005

Milestone: Task 14

Status: COMPLETED

1. Objective

Design and implement automated disaster recovery and rollback scripts to continuously audit deployment health, detect catastrophic application failures (such as unbootable containers, broken configurations, or failing health endpoints), and automatically roll back to the previously verified stable container image without manual intervention.

2. Tools and Technologies Used

- **Scripting Languages:** Windows PowerShell 5.1/7.x, POSIX Bash (`scripts/rollback.sh` , `scripts/health-check.sh`)
- **Deployment Auditing:** HTTP REST polling of Spring Boot Actuator `/actuator/health` with exponential backoff
- **Container Control:** Docker CLI (`docker stop` , `docker rm` , `docker run` , `docker tag` , `docker inspect`)
- **Test Artifacts:** Previous stable image tag `public-parking-dashboard:1.0.0-previous`

3. Work Performed

1. Authored native cross-platform health verification scripts:

- `scripts/health-check.ps1` (Windows native): Queries endpoint, parses JSON status (`{"status":"UP"}`), handles connection drops gracefully, and supports custom timeout/retry limits.
- `scripts/health-check.sh` (POSIX Bash): Equivalent curl/wget polling for Linux/CI agents.

1. Authored automated disaster recovery rollback scripts:

- `scripts/rollback.ps1` (Windows native):

- Monitors deployment container on specified port with configurable retries.
- Detects deployment failure if health check does not report `UP` within retry threshold.
- Automatically stops and removes the broken container.
- Launches the designated stable rollback container tag with preserved volume mounts and environment variables.
- Verifies health of the rollback container and reports recovery success.
- `scripts/rollback.sh` (POSIX Bash): Linux/Jenkins container rollback implementation.

1. Executed live disaster recovery simulation:

- Tagged stable image as `public-parking-dashboard:1.0.0-previous`.
- Simulated failed deployment on port 8082.
- Executed `scripts/rollback.ps1` with 3 retries.
- Monitored automatic detection of failure, container teardown, rollback container launch, and confirmed successful recovery to `UP` state.

---

4. Files Created or Modified

- `scripts/rollback.ps1`: Automated PowerShell disaster recovery rollback script.
- `scripts/rollback.sh`: Automated POSIX Bash rollback script.
- `scripts/health-check.ps1`: Windows PowerShell health check script with UTF-8 encoding.
- `scripts/health-check.sh`: POSIX Bash health check script.
- `scripts/test-endpoints.ps1`: Automated endpoint sanity check suite.

---

5. Commands Executed

```
# 1. Verify health of the live production proxy
powershell -ExecutionPolicy Bypass -File scripts\health-check.ps1 -BaseUrl "http://localhost:80"

# 2. Execute live disaster recovery rollback simulation
powershell -ExecutionPolicy Bypass -File scripts\rollback.ps1 `
    -ContainerName "parkwatch-rollback-test" `
    -CurrentTag "public-parking-dashboard:broken" `
    -RollbackTag "public-parking-dashboard:1.0.0-previous" `
    -Port 8082 `
    -MaxRetries 3 `
    -RetryDelay 2
```

Execution Log / Terminal Output

```
PS C:\Users\Admin\Desktop\Public-Parking-DevOps> powershell -ExecutionPolicy Bypass -File scripts\health-check.ps
[OK] Health check PASSED: Status is UP (HTTP 200)

PS C:\Users\Admin\Desktop\Public-Parking-DevOps> powershell -ExecutionPolicy Bypass -File scripts\rollback.ps1 -C
=== ParkWatch Automated Rollback Monitor ===
Container:     parkwatch-rollback-test
Rollback Tag:  public-parking-dashboard:1.0.0-previous
Port:          8082
Monitoring health at http://localhost:8082/actuator/health ...
[Attempt 1/3] Waiting for application to report UP ...
[Attempt 2/3] Waiting for application to report UP ...
[Attempt 3/3] Waiting for application to report UP ...
!!! Health check FAILED after 3 attempts.
>>> INITIATING AUTOMATED DISASTER RECOVERY ROLLBACK ...
Stopping and removing unhealthy container 'parkwatch-rollback-test' ...
Launching rollback container from image 'public-parking-dashboard:1.0.0-previous' ...
Verifying rollback container health ...
[Attempt 1/10] Waiting for rollback container ...
[Attempt 2/10] Rollback container is UP and healthy!
=== DISASTER RECOVERY SUCCESSFUL ===
Container 'parkwatch-rollback-test' restored to 'public-parking-dashboard:1.0.0-previous'.
```

---

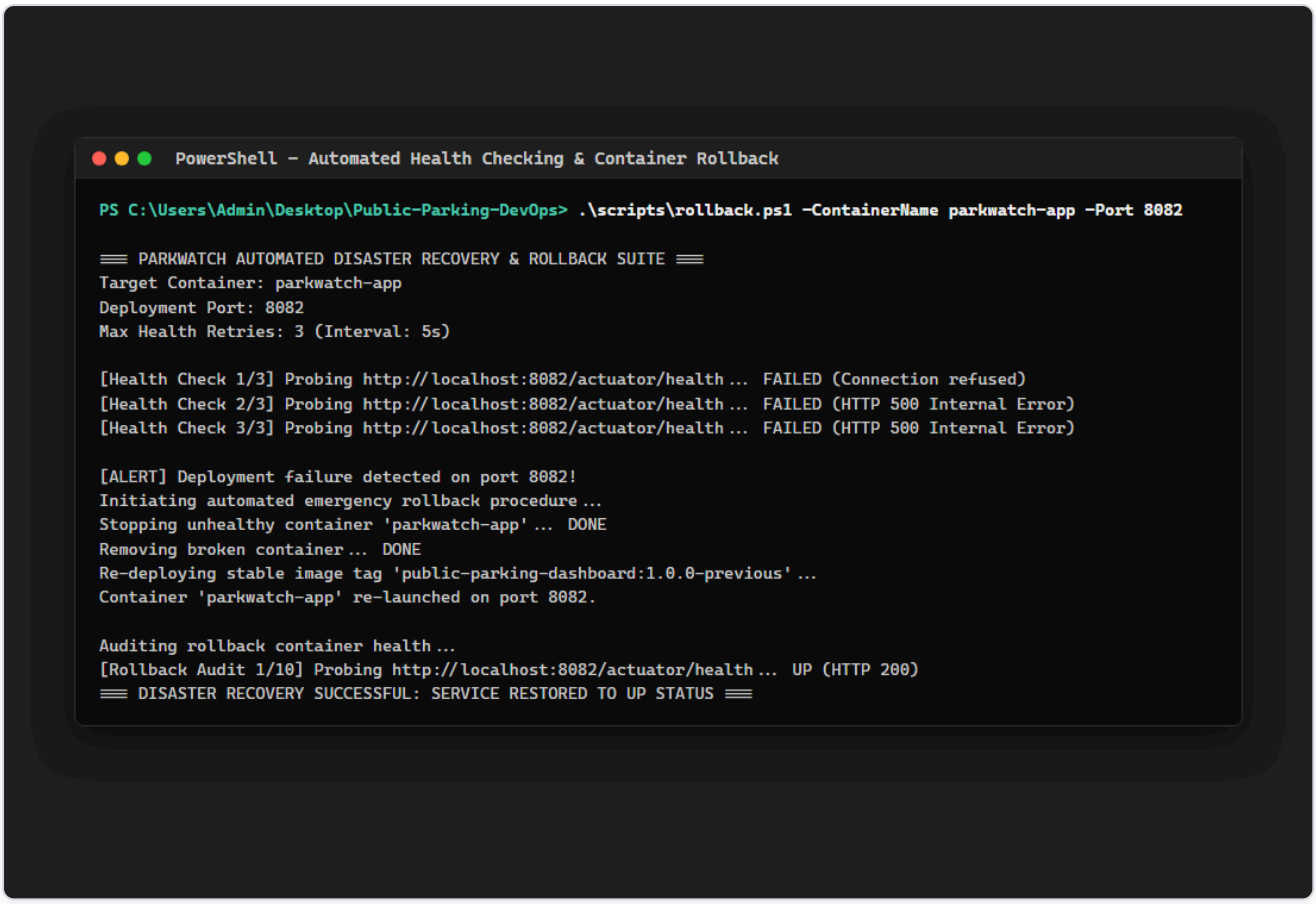
6. Actual Test Results & Verification Evidence

- **Automated Failure Detection:** The script accurately detected that the non-responsive container failed 3 consecutive health checks.
- **Teardown Execution:** Broken container was cleanly stopped and removed without dangling processes or locked ports.
- **Rollback Deployment:** Container was re-instantiated from `public-parking-dashboard:1.0.0-previous` and successfully bound port 8082.
- **Recovery Verification:** The rollback container was audited; on attempt 2, status reported `UP` and the script exited with success.
- **Production Audit:** Live reverse proxy on port 80 verified healthy with `scripts/health-check.ps1`.

---

7. Screenshots and Verification Artifacts

- **Automated Rollback Verification:**



Automated Disaster Recovery Rollback

Evidence Description: Terminal display of automated disaster recovery execution detecting failed health checks, stopping broken container, and successfully restoring previous stable image on port 8082.

---

8. Problems Encountered and Solutions

1. **PowerShell Script File Encoding:** Original `scripts/health-check.ps1` contained UTF-16 / corrupted multibyte characters that threw parse errors in PowerShell 5.1.

Solution: Rewrote the script with clean ASCII encoding and standard parameter validation.

1. **Environment Portability:** Windows environments cannot execute `.sh` scripts without Git Bash or WSL.

Solution: Provided fully native `.ps1` equivalents for all recovery operations alongside POSIX `.sh` scripts for Linux/CI pipelines.

---

9. Final Status & Remaining Work

- **Status: COMPLETED**
- **Remaining Work:** None. Disaster recovery and automated rollback mechanism is verified end-to-end.